

UNIT – 6 LAB EXERCISE

NAME : FURTHOSE SAMREEN B

Reg no: 192511164

TOPIC 6 : BACKTRACKING

1. Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for N = 4, N = 5, and N = 8, showing different valid solutions.

```
1- def solve(n):
2-     board = [-1] * n
3-
4-     def backtrack(r):
5-         if r == n:
6-             print(board)
7-             return True
8-
9-         for c in range(n):
10-            if all(board[i] != c and abs(board[i]-c) != r-i for i in range(r)):
11-                board[r] = c
12-                if backtrack(r + 1):
13-                    return True
14-        return False
15-
16-     backtrack(0)
17-
18- solve(4)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.7 Mb

Your Output

[1, 3, 0, 2]

2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.

```
1- def solve(rows, cols):
2-     board = [-1] * rows
3-
4-     def backtrack(r):
5-         if r == rows:
6-             print(board)
7-             return True
8-
9-         for c in range(cols):
10-            if all(board[i] != c and abs(board[i]-c) != r-i
11-                  for i in range(r)):
12-                board[r] = c
13-                if backtrack(r + 1):
14-                    return True
15-        return False
16-
17-     backtrack(0)
18-
19- solve(8, 10)
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 8.984 Mb

Your Output

[0, 2, 4, 1, 7, 9, 3, 6]

3. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

```

1 def solve(board):
2     for r in range(9):
3         for c in range(9):
4             if board[r][c] == '.':
5                 for n in '123456789':
6                     if (n not in board[r] and
7                         all(board[i][c] != n for i in range(9)) and
8                         all(board[r][j] != n for j in range(9))) and
9                         all(board[c//3*3+i][j//3*3+j] != n for i in range(3) for j in range(3)):
10                    board[r][c] = n
11                    if solve(board):
12                        return True
13                    board[r][c] = '.'
14                return False
15            return True
16
17 board = [
18     ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
19     ["6",".", ".", ".", "1","9",".", ".", ".", "."],
20     [".","9","8",".", ".", ".", ".", ".", "."],
21     ["8",".", ".", "6",".", ".", ".", ".", "3"],
22     ["4",".", ".", "8",".", ".", ".", ".", "1"],
23     [".","7",".", ".", "2",".", ".", ".", "6"],
24     [".","6",".", ".", ".", "2","8",".", "."],
25     [".",".", ".", "4","1","9",".", ".", ".", "5"],
26     [".",".", ".", ".", "8",".", ".", "7","9"]
27 ]
28
29 solve(board)
30 for row in board:
31     print(row)

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 8.784 Mb

Your Output

```

[ '5', '3', '4', '6', '7', '8', '9', '1', '2' ]
[ '6', '7', '2', '1', '9', '5', '3', '4', '8' ]
[ '1', '9', '8', '3', '4', '2', '5', '6', '7' ]
[ '8', '5', '9', '7', '6', '1', '4', '2', '3' ]
[ '4', '2', '6', '8', '5', '3', '7', '9', '1' ]
[ '7', '1', '3', '9', '2', '4', '8', '5', '6' ]
[ '9', '6', '1', '5', '3', '7', '2', '8', '4' ]
[ '2', '8', '7', '4', '1', '9', '6', '3', '5' ]
[ '3', '4', '5', '2', '8', '6', '1', '7', '9' ]

```

4. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

```

1 def solve(board):
2     for r in range(9):
3         for c in range(9):
4             if board[r][c] == '.':
5                 for n in '123456789':
6                     if (n not in board[r] and
7                         all(board[i][c] != n for i in range(9)) and
8                         all(board[r][j] != n for j in range(9))) and
9                         all(board[c//3*3+i][j//3*3+j] != n for i in range(3) for j in range(3)):
10                    board[r][c] = n
11                    if solve(board):
12                        return True
13                    board[r][c] = '.'
14                return False
15            return True
16
17 board = [
18     ["5","3",".", ".", ".", ".", ".", ".", ".", "."],
19     ["6",".", ".", ".", "1","9",".", ".", ".", "."],
20     [".","9","8",".", ".", ".", ".", ".", "."],
21     ["8",".", ".", "6",".", ".", ".", ".", "3"],
22     ["4",".", ".", "8",".", ".", ".", ".", "1"],
23     [".","7",".", ".", "2",".", ".", ".", "6"],
24     [".","6",".", ".", ".", "2","8",".", "."],
25     [".",".", ".", "4","1","9",".", ".", ".", "5"],
26     [".",".", ".", ".", "8",".", ".", "7","9"]
27 ]
28
29 solve(board)
30 for row in board:
31     print(row)

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0400 secs Memory: 8.784 Mb

Your Output

```

[ '5', '3', '4', '6', '7', '8', '9', '1', '2' ]
[ '6', '7', '2', '1', '9', '5', '3', '4', '8' ]
[ '1', '9', '8', '3', '4', '2', '5', '6', '7' ]
[ '8', '5', '9', '7', '6', '1', '4', '2', '3' ]
[ '4', '2', '6', '8', '5', '3', '7', '9', '1' ]
[ '7', '1', '3', '9', '2', '4', '8', '5', '6' ]
[ '9', '6', '1', '5', '3', '7', '2', '8', '4' ]
[ '2', '8', '7', '4', '1', '9', '6', '3', '5' ]
[ '3', '4', '5', '2', '8', '6', '1', '7', '9' ]

```

5. You are given an integer array `nums` and an integer `target`. You want to build an expression out of `nums` by adding one of the symbols `+` and `-` before each integer in `nums` and then concatenate all the integers. For example, if `nums = [2, 1]`, you can add a `+` before 2 and a `-` before 1 and concatenate them to build the expression `"2-1"`. Return the number of different expressions that you can build, which evaluates to `target`.

```

1 def target_sum(nums, target):
2     def backtrack(i, total):
3         if i == len(nums):
4             return total == target
5
6         return (backtrack(i + 1, total + nums[i]) +
7               backtrack(i + 1, total - nums[i]))
8
9     return backtrack(0, 0)
10
11 print(target_sum([1,1,1,1,1], 3))

```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	8.81 Mb

Your Output

5

6. Given an array of integers `arr`, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer modulo $10^9 + 7$.

```

1 def sum_min(arr):
2     total = 0
3
4     for i in range(len(arr)):
5         minimum = arr[i]
6         for j in range(i, len(arr)):
7             minimum = min(minimum, arr[j])
8             total += minimum
9
10    return total % (10**9 + 7)
11
12 print(sum_min([3,1,2,4]))

```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0000 secs	9.072 Mb

Your Output

17

7. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order. The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

```

1 def combination_sum(candidates, target):
2     result = []
3
4     def backtrack(start, target, path):
5         if target == 0:
6             result.append(path[:])
7             return
8
9         for i in range(start, len(candidates)):
10            if candidates[i] > target:
11                continue
12            path.append(candidates[i])
13            backtrack(i, target - candidates[i], path)
14            path.pop()
15
16    backtrack(0, target, [])
17    return result
18
19 print(combination_sum([2,3,6,7], 7))

```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.044 Mb

Your Output

```
[[2, 2, 3], [7]]
```

8. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.

```

1 def combination_sum2(a, target):
2     a.sort()
3     result = []
4
5     def backtrack(start, target, path):
6         if target == 0:
7             result.append(path[:])
8             return
9
10        for i in range(start, len(a)):
11            if i > start and a[i] == a[i-1]:
12                continue
13            if a[i] > target:
14                break
15
16            path.append(a[i])
17            backtrack(i + 1, target - a[i], path)
18            path.pop()
19
20    backtrack(0, target, [])
21    return result
22
23 print(combination_sum2([10,1,2,7,6,1,5], 8))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	9.124 Mb

Your Output

```
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
```

9. Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

```
1 def permutations(nums):
2     result = []
3
4     def backtrack(path, remaining):
5         if not remaining:
6             result.append(path[:])
7             return
8
9         for x in remaining:
10             backtrack(path + [x], [y for y in remaining if y != x])
11
12     backtrack([], nums)
13     return result
14
15 print(permutations([1,2,3]))
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.916 Mb

Your Output

[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

10. Given a collection of numbers, `nums`, that might contain duplicates, return all possible unique permutations in any order.

```
1 def permutations(nums):
2     nums.sort()
3     result = []
4
5     def backtrack(path, used):
6         if len(path) == len(nums):
7             result.append(path[:])
8             return
9
10        for i in range(len(nums)):
11            if used[i] or (i > 0 and nums[i] == nums[i-1] and not used[i-1]):
12                continue
13
14            used[i] = True
15            path.append(nums[i])
16            backtrack(path, used)
17            path.pop()
18            used[i] = False
19
20    backtrack([], [False] * len(nums))
21    return result
22
23 print(permutations([1,1,2]))
```

Ctrl + / Copy Paste Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 9.04 Mb

Your Output

[[1, 1, 2], [1, 2, 1], [2, 1, 1]]

11. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally

color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4

```

1- def graph_coloring(n, edges, k):
2-     graph = [[] for _ in range(n)]
3-
4-     for u, v in edges:
5-         graph[u].append(v)
6-         graph[v].append(u)
7-
8-     color = [0] * n
9-
10-    def valid(v, c):
11-        return all(color[x] != c for x in graph[v])
12-
13-    def backtrack(v):
14-        if v == n:
15-            return True
16-
17-        for c in range(1, k + 1):
18-            if valid(v, c):
19-                color[v] = c
20-                if backtrack(v + 1):
21-                    return True
22-                color[v] = 0
23-
24-        return False
25-
26-    return backtrack(0), color
27-
28-edges = [(0,1),(1,2),(2,3),(3,0),(0,2)]
29-print(graph_coloring(4, edges, 3))

```

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.708 Mb

Your Output

(True, [1, 2, 3, 2])

12. You and your friends are tasked with coloring a map using a limited set of colors, with the following rules: At each step, you can choose any region of the map that hasn't been colored yet and color it with any available color. Your friend Alice will then color the next region using the same strategy, followed by your friend Bob. You aim to maximize the number of regions you color. Given a map represented as a list of regions and their adjacency relationships, write a function to determine the maximum number of regions you can color. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4, k = 3

```

1- def graph_coloring(n, edges, k):
2-     graph = [[] for _ in range(n)]
3-
4-     for u, v in edges:
5-         graph[u].append(v)
6-         graph[v].append(u)
7-
8-     color = [0] * n
9-
10-    def valid(v, c):
11-        for x in graph[v]:
12-            if color[x] == c:
13-                return False
14-        return True
15-
16-    def solve(v):
17-        if v == n:
18-            return True
19-
20-        for c in range(1, k + 1):
21-            if valid(v, c):
22-                color[v] = c
23-                if solve(v + 1):
24-                    return True
25-                color[v] = 0
26-
27-        return False
28-
29-    if solve(0):
30-        return color
31-    return None
32-
33-edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
34-print(graph_coloring(4, edges, 3))

```

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.704 Mb

Your Output

[1, 2, 3, 2]

13. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and $n = 5$

```

1 def hamiltonian(n, edges):
2     graph = [[] for _ in range(n)]
3
4     for u, v in edges:
5         graph[u].append(v)
6         graph[v].append(u)
7
8     path = [0]
9
10    def backtrack():
11        if len(path) == n:
12            return path[0] in graph[path[-1]]
13
14        for v in range(1, n):
15            if v not in path and v in graph[path[-1]]:
16                path.append(v)
17                if backtrack():
18                    return True
19                path.pop()
20
21        return False
22
23    return backtrack()
24
25 edges = [(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
26 print(hamiltonian(5, edges))

```

Enter Input here

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.872 Mb

Your Output

False

14. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and $n = 4$

```

1 def hamiltonian(n, edges):
2     graph = [[] for _ in range(n)]
3
4     for u, v in edges:
5         graph[u].append(v)
6         graph[v].append(u)
7
8     path = [0]
9
10    def solve():
11        if len(path) == n:
12            return path[0] in graph[path[-1]]
13
14        for v in range(1, n):
15            if v not in path and v in graph[path[-1]]:
16                path.append(v)
17
18                if solve():
19                    return True
20
21                path.pop()
22
23        return False
24
25    if solve():
26        return path + [0]
27    return None
28
29 edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
30 print(hamiltonian(4, edges))

```

Ctrl+Shift+Enter

If your code takes input, add it in the above box before

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.792 Mb

Your Output

[0, 1, 2, 3, 0]

15. You are tasked with designing an efficient coding to generate all subsets of a given set S containing n elements. Each subset should be outputted in lexicographical order. Return a list of lists where each inner list is a subset of the given set. Additionally, find out how your coding handles duplicate elements in S. A = [1, 2, 3] The subsets of [1, 2, 3] are: [], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]

```

1 def subsets(a):
2     result = []
3
4     def backtrack(i, path):
5         if i == len(a):
6             result.append(path[:])
7             return
8
9         backtrack(i + 1, path)
10
11        path.append(a[i])
12        backtrack(i + 1, path)
13        path.pop()
14
15    backtrack(0, [])
16    return result
17
18 print(subsets([1,2,3]))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.576 Mb

Your Output

```

[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]

```

16. Write a program to implement the concept of subset generation. Given a set of unique integers and a specific integer 3, generate all subsets that contain the element 3. Return a list of lists where each inner list is a subset containing the element 3 E = [2, 3, 4, 5], x = 3, The subsets containing 3 : [3], [2, 3], [3, 4], [3,5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5] Given an integer array nums of unique elements, return all possible subsets(the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

```

1 def subsets(a, x):
2     result = []
3
4     def backtrack(i, path):
5         if i == len(a):
6             if x in path:
7                 result.append(path[:])
8                 return
9
10        backtrack(i + 1, path)
11
12        path.append(a[i])
13        backtrack(i + 1, path)
14        path.pop()
15
16    backtrack(0, [])
17    return result
18
19
20 print(subsets([2, 3, 4, 5], 3))

```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0200 secs Memory: 8.656 Mb

Your Output

```

[[3], [3, 5], [3, 4], [3, 4, 5], [2, 3], [2, 3, 5], [2, 3, 4], [2, 3, 4, 5]]

```

17. You are given two string arrays words1 and words2. A string b is a subset of string a if every letter in b occurs in a including multiplicity. For example, "wrr" is a subset of "warrior" but is not a subset of "world". A string a from words1 is universal if for every string b in words2, b is a subset of a. Return an array of all the universal strings in words1. You may return the answer in any order.


```
1 from collections import Counter
2
3 def word_subsets(words1, words2):
4     need = Counter()
5
6     for word in words2:
7         c = Counter(word)
8         for ch in c:
9             need[ch] = max(need[ch], c[ch])
10
11     result = []
12
13     for word in words1:
14         if all(counter(word)[ch] >= need[ch] for ch in need):
15             result.append(word)
16
17     return result
18
19 words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
20 words2 = ["e", "o"]
21
22 print(word_subsets(words1, words2))
```

Enter Input here

If your code takes input, add it in the above box before running

Output

Status : Successfully executed

Time:

0.0200 secs

Memory:

9.796 Mb

Your Output

['facebook', 'google', 'leetcode']